FOODDASH - ANDROID STUDIO APK COMPILATION & LAB REPORT GUIDE

Course: Mobile Application and Development Lab (0613-410) | Package: com.fooddash.app

Stack: 100% Native Java + XML Layouts | Backend: Firebase Auth, Realtime DB & Storage

SECTION 1: FoodDash Android Studio Project Structure (Java + XML)

Package: com.fooddash.app | Target SDK: 34 | Minimum SDK: 24 | Language: Java 17

FoodDash is architected strictly as a native Android Studio project using Java for all business logic and XML for all user interface layouts. No Kotlin application source code, Jetpack Compose, or hybrid web frameworks are used in the Android source tree.

Before compiling the APK using either Method 1 (Android Studio) or Method 2 (GitHub Actions), ensure your extracted FoodDash/ directory contains the following canonical structure:

1.1 Root Gradle & Project Configuration Files

Located at the root of the extracted FoodDash/ folder. These files configure Gradle 8.7, the Android Gradle Plugin (AGP 8.5.2), Google Services plugin (4.4.2), and AndroidX properties.

```
FoodDash/
|-- build.gradle.kts                # Root project plugins (AGP 8.5.2, google-services 4.4.2)
|-- settings.gradle.kts             # Configures Google & MavenCentral repos, includes :app
|-- gradle.properties               # Enables AndroidX & sets JVM heap (-Xmx2048m)
|-- database.rules.json             # Firebase Realtime Database security rules
|-- README.md                       # University submission documentation
|-- .github/workflows/build-apk.yml # Automated GitHub Actions Cloud APK Builder
```

1.2 Application Module (app/) & Java Packages

All 25 Java files are organized into 4 modular sub-packages under app/src/main/java/com/fooddash/app/ alongside 17 XML layout files under app/src/main/res/layout/.

```
FoodDash/app/
|-- build.gradle.kts                # Module config (com.fooddash.app, Firebase BoM 33.5.1, Glide
|-- google-services.json            # Firebase configuration file (starter template included)
|-- src/main/
    |-- AndroidManifest.xml         # Declares 14 Activities, permissions, & launcher intent
    |-- java/com/fooddash/app/
    |   |-- activities/             # All 14 Screen Activities (Splash, Login, CustomerHome, Admi
    |   |-- models/                 # User.java, Food.java, Order.java, CartItem.java
    |   |-- adapters/               # FoodAdapter.java, CartAdapter.java, OrderAdapter.java
    |   |-- helpers/                # FirebaseHelper.java, SessionManager.java, ValidationHelper.
    |-- res/
        |-- layout/                 # 14 Activity XML files + 3 RecyclerView item XML files
        |-- drawable/               # ic_fooddash_logo.xml custom vector icon
        |-- values/                 # colors.xml, strings.xml, themes.xml
```

SECTION 2: Firebase Configuration & Security Rules Setup

Connecting Firebase Authentication, Realtime Database, and Storage to com.fooddash.app

The exported FoodDash project includes a valid starter app/google-services.json file so that Gradle can compile a debug APK immediately without throwing a missing-file error. To connect the compiled APK to your live Firebase cloud backend for your lab demonstration, follow these steps:

Step 1: Create Project & Register Android Package

Open https://console.firebase.google.com/, click "Add project", and name it FoodDash.

Click the Android icon to register a new app using the exact package name com.fooddash.app.

Step 2: Replace app/google-services.json

Download the generated google-services.json file from Firebase Console and overwrite FoodDash/app/google-services.json.

Step 3: Enable Email/Password Authentication

In Firebase Console, navigate to Build -> Authentication -> Sign-in method, select Email/Password, toggle Enable, and click Save.

Step 4: Provision Realtime Database & Publish Security Rules

Navigate to Build -> Realtime Database -> Create Database. Open the Rules tab and paste the contents of database.rules.json. This rule ensures customers cannot self-assign the ADMIN role.

```
"users": {
  "$uid": {
    ".read": "auth != null && (auth.uid === $uid || root.child('fooddash/users').child(auth.uid)
    ".write": "auth != null && auth.uid === $uid && (!newData.child('role').exists() || newData.
  }
}
```

Step 5: Provision Restaurant Admin Account

Register a normal user account in the app, then open Firebase Realtime Database console at fooddash/users/{uid}/role and change the value from "CUSTOMER" to "ADMIN". When that user logs in on LoginActivity, FoodDash automatically routes them to AdminDashboardActivity.

SECTION 3: Method 1: Manual APK Compilation in Android Studio

Standard Lab Procedure for Compiling, Debugging, and Installing app-debug.apk on a Local PC

Use Method 1 when working on your personal laptop or university lab PC with Android Studio installed. This workflow compiles the Java 17 source code and XML resources into an installable Android Package (.apk).

Step 1: Export & Extract the FoodDash Project Archive

Click "Export Android Project" in the FoodDash web workspace to download FoodDash-AndroidStudio-Java-XML.tar. Extract the archive using WinRAR, 7-Zip, macOS Archive Utility, or terminal:

```
tar -xvf FoodDash-AndroidStudio-Java-XML.tar
```

Step 2: Open Project in Android Studio & Configure JDK 17

Launch Android Studio (Koala / Ladybug or newer), select File -> Open, and choose the extracted FoodDash/ directory. Verify that File -> Settings -> Build, Execution, Deployment -> Build Tools -> Gradle -> Gradle JDK is set to JDK 17 (jbr-17).

Step 3: Run Gradle Sync

Click "Sync Project with Gradle Files" (the elephant icon in the top toolbar). Gradle will download AndroidX AppCompat, Material Components 1.12.0, ConstraintLayout, RecyclerView, CardView, Firebase BoM 33.5.1, Glide 4.16.0, and Gson 2.11.0.

Step 4: Trigger Build APK(s) from the Menu or Terminal

In the Android Studio top menu bar, select Build -> Build Bundle(s) / APK(s) -> Build APK(s). Alternatively, run the Gradle assembleDebug task in the Android Studio Terminal:

```
./gradlew assembleDebug    # On macOS / Linux
gradlew.bat assembleDebug  # On Windows
```

Step 5: Locate & Install the Generated APK on Your Phone

Once the build finishes ("BUILD SUCCESSFUL"), click "locate" in the bottom-right notification balloon or navigate to the output path below. Enable "Install unknown apps" on your Android phone and tap the APK to install, or install directly via USB Debugging:

```
# Output APK Location:
FoodDash/app/build/outputs/apk/debug/app-debug.apk

# Optional Direct USB Installation via ADB:
adb install -r app/build/outputs/apk/debug/app-debug.apk
```

SECTION 4: Method 2: Automated Cloud APK Build via GitHub Actions

Zero-Local-Setup Compilation Using .github/workflows/build-apk.yml & GitHub Release v1.0

If your personal computer has limited RAM or trouble running Android Studio locally, Method 2 compiles the exact same Java + XML project on GitHub's cloud servers (ubuntu-latest) automatically whenever you push code to your repository.

This method also directly fulfills Section 3 of the Final Lab Project instructions: maintaining a public GitHub repository with regular commits and attaching the compiled APK to GitHub Release v1.0.

Step 1: Initialize Git & Push FoodDash to a Public GitHub Repository

Create a new public repository named FoodDash-Android on GitHub. Open a terminal inside the extracted FoodDash/ directory and push all 55 project files:

```
cd FoodDash
git init
git add .
git commit -m "Feat: Complete FoodDash Java + XML Android Studio project with 14 Activities"
git branch -M main
git remote add origin https://github.com/<your-username>/FoodDash-Android.git
git push -u origin main
```

Step 2: Understand the Included .github/workflows/build-apk.yml Pipeline

The exported project includes .github/workflows/build-apk.yml. As soon as git push completes, GitHub Actions provisions an Ubuntu runner, installs Eclipse Temurin JDK 17, configures Gradle 8.7, executes gradle assembleDebug, and uploads the compiled APK as a downloadable artifact:

```
name: Build FoodDash Android APK
on:
  push:
    branches: [ "main", "master" ]
  workflow_dispatch:
jobs:
  build-apk:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
      - uses: gradle/actions/setup-gradle@v3
        with:
          gradle-version: '8.7'
      - run: gradle assembleDebug --stacktrace
      - uses: actions/upload-artifact@v4
        with:
```

```
          name: FoodDash-v1.0-APK
          path: app/build/outputs/apk/debug/app-debug.apk
```

Step 3: Download the Compiled APK from GitHub Actions

1. Open your GitHub repository in any browser (on PC or mobile).
2. Click the "Actions" tab at the top.
3. Click the latest green workflow run ("Build FoodDash Android APK").
4. Scroll down to the "Artifacts" section and click "FoodDash-v1.0-APK" to download the ZIP containing app-debug.apk.

Step 4: Publish GitHub Release v1.0 (Required Lab Deliverable)

1. On your GitHub repository main page, click "Releases" -> "Create a new release" in the right sidebar.
2. Click "Choose a tag", type v1.0, and select "Create new tag: v1.0 on publish".
3. Set Release Title to "FoodDash v1.0 - Final Lab Submission".
4. Drag and drop your compiled FoodDash-v1.0.apk (renamed from app-debug.apk) into the binaries box.
5. Click "Publish release".

SECTION 5: Post-Installation Testing Checklist & Viva Defense Summary

Verifying All 12 Mandatory University Lab Features on Your Physical Android Device

After installing FoodDash-v1.0.apk on your Android smartphone, walk through this 6-point checklist before your final lab demonstration and viva:

Check 1: Input Validation & Firebase Authentication (Reqs #1, #3, #7, #9)

Tap Sign In on LoginActivity with an empty email to demonstrate TextInputEditText.setError(). Then register a new Customer account on RegisterActivity, verify "Remember Me" persistence in SharedPreferences by closing and reopening the app via SplashActivity.

Check 2: RecyclerView Menu, Categories & Search (Reqs #2, #4, #5, #8)

On CustomerHomeActivity, filter dishes by Burgers / Biryani / Pizza, search by keyword, and tap a dish to open FoodDetailsActivity via Explicit Intent with Serializable Food extra.

Check 3: Cart & Manual bKash / Nagad Payment Reference (Section 5)

Add items to CartActivity, adjust quantities, proceed to CheckoutActivity, select bKash or Nagad, observe the displayed merchant number, enter a 6-20 character alphanumeric Transaction ID (e.g., BKA948271X), and place the order. Confirm that paymentStatus is saved as PENDING_VERIFICATION.

Check 4: Hardware Accelerometer Shake-to-Refresh (Req #11)

Physically shake your Android phone on CustomerHomeActivity or MyOrdersActivity. ShakeDetector.java computes gForce = sqrt(gX^2 + gY^2 + gZ^2) > 2.7g and refreshes the Firebase Realtime Database listener with a Toast confirmation.

Check 5: Camera / Gallery Intent & Implicit Intents (Reqs #10, #12)

Open ProfileActivity to capture a photo with the Camera or select from Gallery. Test the Call Support button (ACTION_DIAL), Share Dish/Receipt button (ACTION_SEND), and Website button (ACTION_VIEW).

Check 6: Admin Dashboard, Food CRUD & Payment Verification (Reqs #6, #8)

Log into the Restaurant Admin account to view AdminDashboardActivity (Total Orders, Pending, Completed, Total Foods, Sales Summary). Open ManageFoodActivity to add a dish via

FloatingActionButton or delete a dish via AlertDialog. Open AdminOrdersActivity to verify the customer’s bKash Transaction ID (PENDING_VERIFICATION -> VERIFIED) and update order status to DELIVERED.